

Faculty of Electrical Engineering – University of Sarajevo

Course: Artificial Intelligence

Professor: Izudin Džafić

Team: Rijalda Šaćirbegović, Almir Mustafić, Tarik Bajrović

PatternScope AI

Hybrid Pattern Recognition System

Executive Summary

PatternScope AI is a hybrid pattern recognition system that supports the digitization of handwritten academic material. The current implementation recognizes handwritten digits, geometric shapes, and custom symbols. Users can upload images. The system returns per-model predictions, confidence scores, and an ensemble result. Users can also provide corrections during runtime.

The project focuses on the first part of a larger workflow. Students, teaching assistants, and professors often prepare notes, exam materials, lab reports, formulas, and diagrams by hand. Converting these materials into digital form takes time and may require several software tools. The project uses pattern recognition as the first step toward converting this material into reusable digital content.

The planned platform will assemble recognized elements into text, equations, and vector drawings. It will export content to editable PDF and LaTeX formats. The project proposes hosting the platform at the University of Sarajevo. Students, teaching assistants, and professors at Bosnian universities would use the platform without subscription fees. The hosting model would keep academic material on local university servers rather than third-party clouds.

1. About the Project

PatternScope AI currently performs three recognition tasks.

- The system recognizes handwritten digits from 0 to 9.
- The system recognizes circles, squares, and triangles.
- The system recognizes checkmarks, crosses, and hearts.

Users can upload images. The system returns predictions, confidence scores, and model comparisons. When users provide the correct label, the system updates its internal models during runtime.

The next phase will use the same recognition engine as a base for the following functions.

- The platform will assemble recognized elements into text.
- The platform will assemble recognized elements into equations.
- The platform will assemble recognized elements into vector drawings.
- The platform will export content to editable PDF and LaTeX formats.

2. System Architecture Overview

PatternScope AI is built as a modular hybrid AI engine composed of:

- Data processing pipeline
- Multiple supervised learning models
- An ensemble decision mechanism
- A rule-based fallback system
- Real-time active learning agent
- GUI interface via natID framework

The backend logic is implemented in C++14 and exposed to the GUI through a central Engine class.

3. Technology Stack

The system is implemented using:

- **C++14** – Core AI engine and model implementation
- **CMake** – Cross-platform build management
- **stb_image** – Image loading and preprocessing
- **natID GUI framework** – Interactive desktop interface

The system is fully offline and cross-platform compatible.

4. Datasets

The system is trained on three datasets:

4.1 MNIST Dataset

- 60,000 training images
- 10,000 test images
- 28×28 grayscale digit images
- IDX file format

4.2 Quick Draw Shapes Dataset

- 1,000 training samples per class
- 200 test samples per class
- Classes: circle, square, triangle

4.3 Quick Draw Symbols Dataset

- 1,000 training samples per class
- 200 test samples per class
- Classes: checkmark, cross, heart

All images are:

- Resized to 28×28
- Normalized to [0, 1]
- Converted into feature vectors

Training is performed using a standard train-test split.

5. Algorithms & Models

PatternScope AI uses a hybrid ensemble composed of five components:

- K-Nearest Neighbors (KNN)
- Naive Bayes
- Mini Multi-Layer Perceptron (MLP)
- A* Template Matcher
- Rule-Based Engine (Fallback)

Each model contributes unique strengths in classification, explainability, and robustness.

6. How the Algorithms Work

6.1 K-Nearest Neighbors (KNN)

KNN stores all training feature vectors.

During prediction:

Euclidean distance is computed between input and stored samples.

The 3 nearest neighbors are selected.

The most frequent label among neighbors is returned.

Confidence is computed based on neighbor agreement and distance.

Strength:

Immediate adaptability through active learning.

No parameter retraining required.

6.2 Naive Bayes

Naive Bayes models each pixel using a Gaussian distribution.

During training:

Mean and variance are computed per pixel per class.

During prediction:

Class likelihood is computed using Gaussian probability.

The class with maximum posterior probability is selected.

Strength:

Fast probabilistic inference.

Works well with grayscale intensity distributions.

6.3 Mini Multi-Layer Perceptron (MLP)

Architecture:

- 784 input neurons (28×28 pixels)
- 64 hidden neurons
- Output neurons depend on task (10 for digits, 3 for shapes/symbols)

Training:

- Random weight initialization (-0.5 to 0.5)
- 5 epochs
- Learning rate: 0.01
- Backpropagation for gradient updates

Strength:

- Captures nonlinear pixel relationships.
- Higher accuracy for complex patterns.

6.4 A* Template Matcher

Uses first 200 training samples as templates.

Prediction process:

- Compute Euclidean distance to each template.
- Use priority queue for sorting.
- Return closest match.

Strength:

Deterministic

Fully explainable

Allows template visualization

Unlike neural networks, A* provides transparent reasoning.

6.5 Rule-Based Engine

Triggered when ensemble confidence < 0.4.

Uses structural features:

- Edge count
- Corner count
- Aspect ratio
- Symmetry

Applies deterministic rules such as:

“4 corners + rectangular geometry → likely 0 or 8”

Purpose:

- Prevent silent failures
- Ensure fallback decision path

7. Ensemble Decision Mechanism

Each model returns:

Predicted label

Confidence score

Confidence aggregation

Scores per label are summed

Highest total score wins

Final confidence = total score / 4.0

If final confidence < 0.4:

→ Rule-Based Engine overrides decision.

This ensures:

Robust decision-making

Honest uncertainty handling

Multi-model agreement verification

8. Active Learning Mechanism

When user provides correct label:

- KNN appends new feature vector
- Naive Bayes updates mean & variance incrementally
- MLP performs one forward + backward update

These updates occur in RAM during runtime.

This enables:

- Incremental adaptation
- Institution-specific handwriting learning
- Continuous system improvement

9. Model Training & Serialization

Training is performed offline using:

`./trainer`

After training:

Models are serialized into:

- knn.txt
- nb.txt
- mlp.txt
- astar.txt

At runtime:

Models are loaded into memory

No retraining required

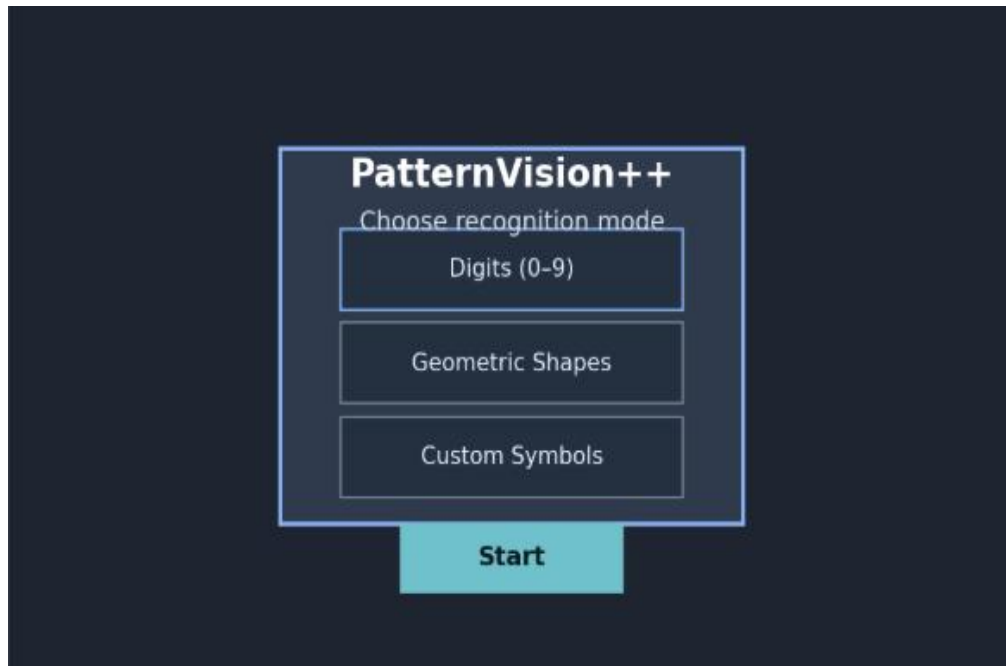
This reduces startup latency and ensures deployment efficiency.

10. Graphical User Interface (PatternVision++)

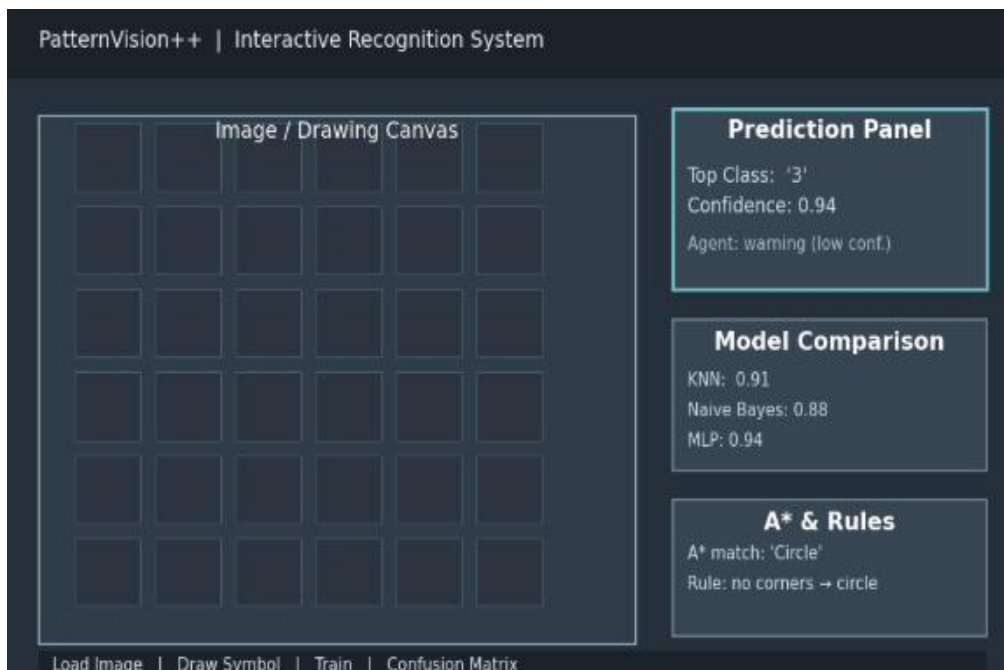
The natID-based GUI allows:

- Mode selection (Digits, Shapes, Symbols)

- Image upload
- Real-time predictions
- Model comparison panel
- Confidence visualization
- Active learning feedback
- Confusion matrix & metrics view



Caption: Figure 1 – PatternVision++ Mode Selection Interface



Caption: Figure 2 – Live Prediction and Model Comparison Panel

11. Problem and Market Gap

Every semester, students and staff create exam materials, lab reports, notes, formulas, diagrams, and geometric drawings by hand. Converting this material into digital form takes time. Using several software tools can shift attention from course content to software use. Paper-based notes are also difficult to reuse across new groups of students. The team reviewed existing tools while defining the project. Microsoft 365 and Mathpix use recurring subscription models. Apple Math Notes offers a free option but requires Apple devices. The project therefore targets a gap for a device-independent academic platform that can be used across Bosnian universities.

12. Target Users

- Students can digitize notes, equations, and study materials.
- Teaching assistants can digitize exercises, diagrams, and lab materials.
- Professors can convert handwritten course material into editable content.
- Universities can store and reuse academic material across courses and student generations.

13. Roadmap and Hosting

The current phase recognizes individual digits, shapes, and symbols. The next phase will assemble recognized elements into text, equations, and vector drawings. The platform will export content to editable PDF and LaTeX formats.

The project proposes hosting the platform at the University of Sarajevo. Students, teaching assistants, and professors at Bosnian universities would use the platform without subscription fees. The hosting model would keep academic work and research notes on local university servers rather than third-party clouds.